



Week 2

- Requirement Engineering Part 1 & React**



Requirement Engineering - Intro

- First step in the software.
- It's challenging, costly and could be riskiest step of them all.
- Requirement engineer could be called:
 - Software Analyst
 - System Analyst
 - Process Analyst
 - Business Architect
 - Product Manager
 - Product Owner
 - Consultant



Contd.

Software Req
fundamentals

Requirements
Process model

Requirements
Elicitation

Requirements
Analysis

Requirements
documentation

Requirements
Validation

Practical
Consideration

Requirements
Tools



Without Requirements

- Developers won't know what is considered "Complete"
- Testers won't know what to test.
- Customers don't know what to expect
- Users can't determine if the software / product will meet their needs



With Invalid Requirements

- Can lead to non-useful system.
- Lead to major changes , schedule slippage which will result in higher cost.
- Lead to extra steps to perform simple tasks using the software , making the system harder to use.
- Can lead to unnecessarily complicated unusable system.



Requirement Elicitation

- Requirement elicitation is concerned with origin of software requirement and how the software engineer can discover them.
- Other term for this activity is “Requirement Gathering”
 - Gathering means the requirement are already exists and ready for collection & validation
- “Elicitation” is drawing out, calling forth or evoking a response or reaction
- Also termed as “requirement capture”, “requirement discovery”, and “requirement acquisition”
- First stage in understanding the problem the software / product is required to solve
- Establish the relationships with stakeholder



Issues & Problems in Requirement Elicitation

- Stakeholder might skip information thinking it's obvious enough, "it goes without saying"
- Communication gap between analysts and stakeholders
- User tells you the solution to their problem , not the problem itself
- Stakeholders get bored from the meeting and the questions

Requirement Sources

- Requirements have many sources in the typical software and all possible sources must be recognized and evaluated.
- There are 6 primary sources of requirements.

Goals

Domain
Knowledge

Stakeholders

Business
Rules

Operational
Env.

Organization
Environment



Interview

- One of the most popular technique used by Business Analysts
- Low-cost conversational technique
- One to one, one to many , many to many or many to one
- Asking relevant question and documenting
- Used to establish relationships and built trust with stakeholders.



Popular Questions and Active Listening

- Ask your interviewee to repeat the central ideas, theme, and summaries to assure complete understanding
- Ask open-ended questions such as:
 - What could prevent user from successfully completing a task.
 - How should system respond to various error conditions
 - What else could
 - Would you ever need
- Key factors in deciding what to ask:
 - Understand the person you are interviewing
 - Knowing what you want to achieve as a result of the discussion
 - Understanding the whole project context.
 - Asking “Why” several times
- Do not draw conclusion until you have heard the whole story
- Understanding is not agreeing
- Do not be afraid to ask if there's something that you have not been told

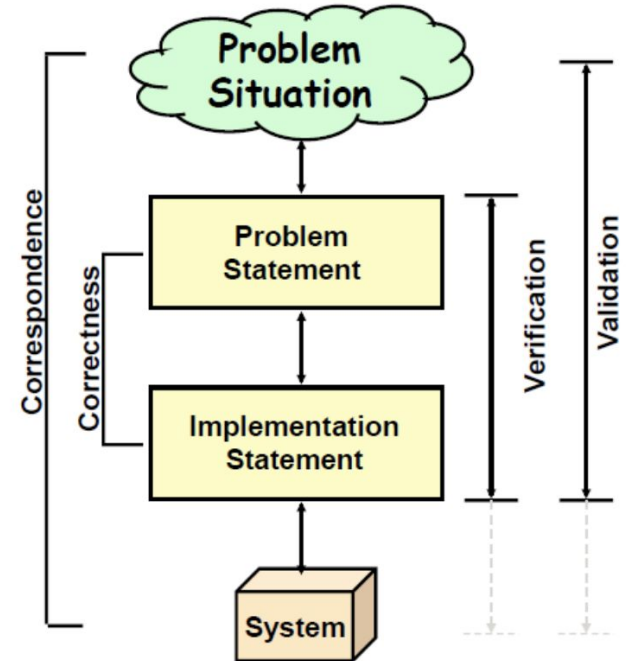
Separating Problem from Solution

A Separate problem description is useful

- It can be discussed with stakeholders
- It can be used to evaluate design choices - It is a good source of test cases

Still need to check

- Solution correctly solves the stated problem
- Problem statement corresponds to the needs of the stakeholders (validation)





Identify Stakeholders' Goal

Approach

- Focus on **why** a system is required
- Express the “why” as a set of stakeholder goals
- Use goal refinement to arrive at specific requirements
- Goal analysis: document, organize and classify goals
- Goal evolution: refine, elaborate, and operationalize goals - Goal hierarchies show refinements and alternatives

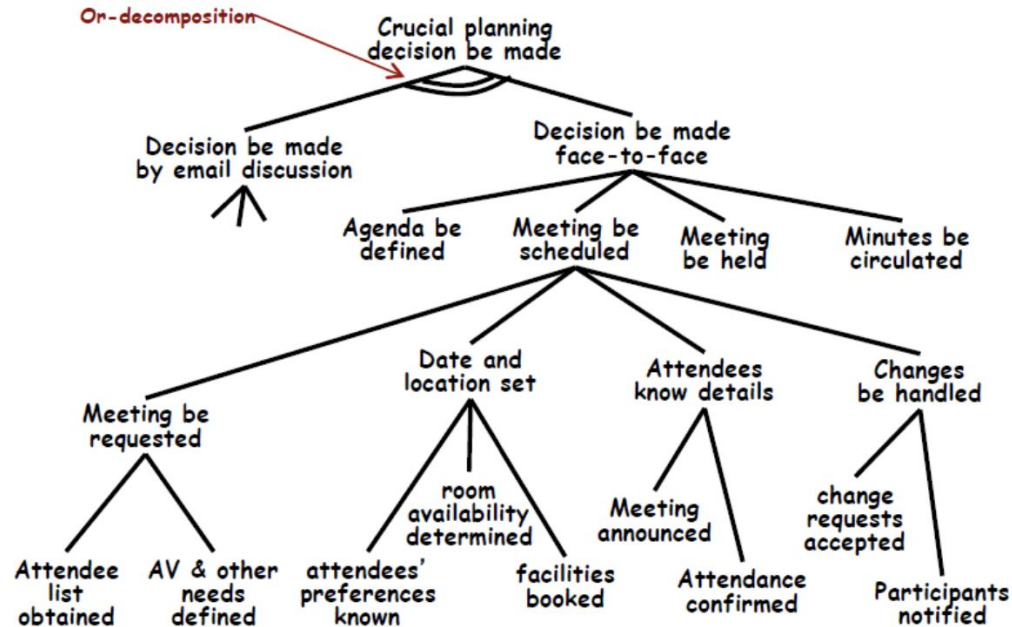
Advantages

- Reasonably intuitive
- Explicit declaration of goals provides sound basis for conflict resolution

Disadvantages

- Captures A Static Picture-what if goals change over time?
- Can regress forever up (or down) the goal hierarchy

Goal Elaboration



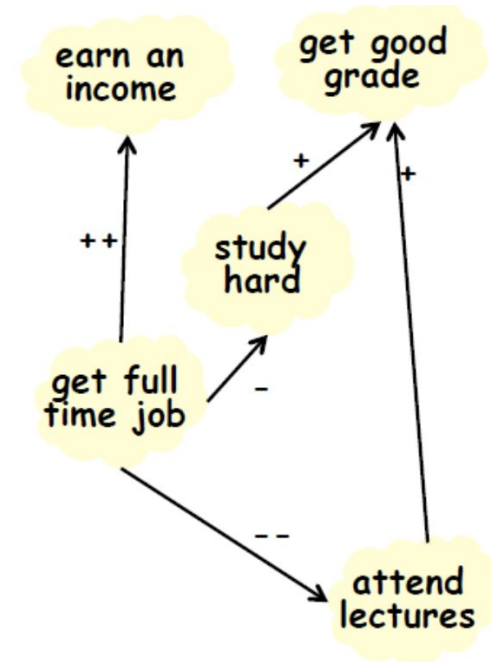
Goal Analysis

Goal Elaboration:

- “Why” questions explore higher goals (context)
- “How” questions explore lower goals (operations)
- “How else” questions explore alternatives

Relationship between goals:

- One goal helps achieve another (+)
- One goal hurts achievement of another (-)
- One goal makes another (++)
- One goal breaks another (--)

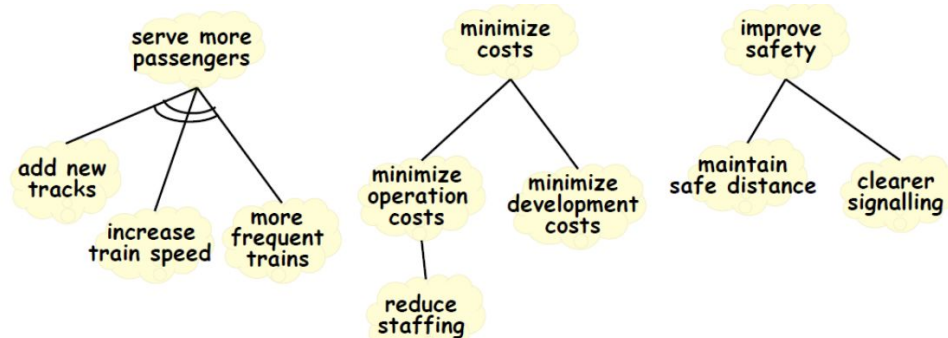


Soft Goals

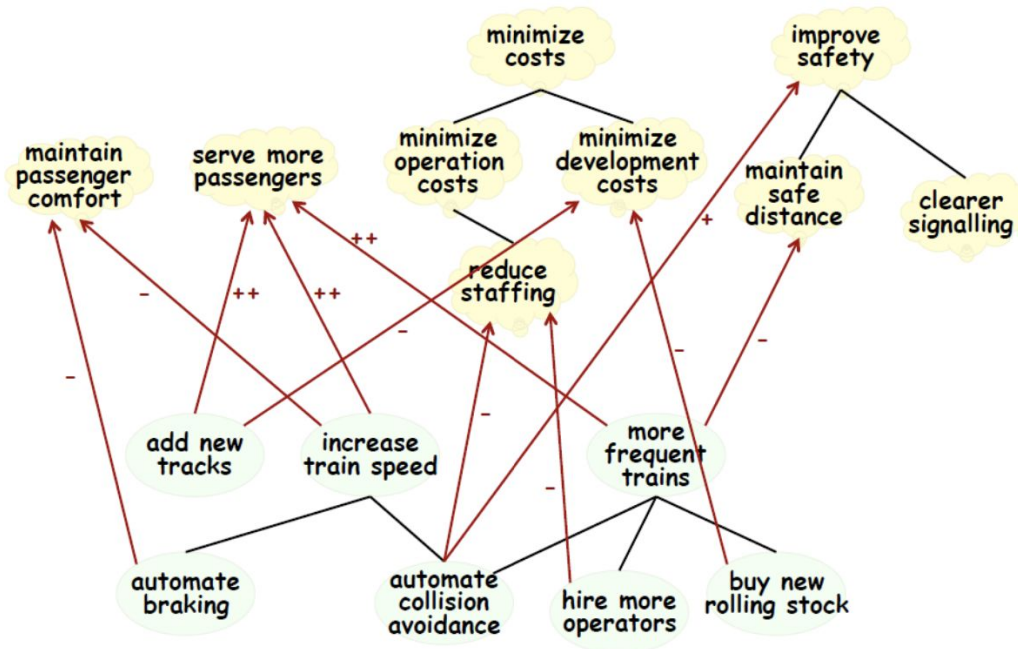
Some goals may be subjective:

- Often related to qualitative aspects such as usability, performance, etc
- Treat these as soft goals
E.g. “system be easy to use”, “access be secure”

Train System Example:



Soft Goal as Selection Criteria





Prototypes

- A prototype is an early sample , model or release of a product built to test a concept or process
- Prototype can act as scenarios to users, providing better understanding of their needs
- Valuable tool for clarifying ambiguous requirement
- Give the users the easiest opportunity to imagine what final product would look like

- 
- Prototype typically simulates various aspects of the actual final product such as functionality, user, interface and workflow
 - Two most famous type of prototype:
 - Horizontal Prototype:
 - Focus on one layer of the system
 - Provides a broad view of an entire system focussing on user interaction
 - Build the system breadth-first then depth
 - Vertical Prototype:
 - An enhanced complete elaboration of a single system of function
 - Each section is built in detail with additional iterations detailing subsequent sections
 - It's useful for obtaining detailed requirements for given function



Categories of Prototype (DSDM)

An agile methodology called Dynamic System Development Method (DSDM) recommend four categories of prototype

Business Prototypes: Design and demonstrate the business process being automated.

Usability Prototypes: Define , refine , and demonstrate user interface design usability, accessibility , look and feel

Performance and Capacity Prototypes: Define, demonstrate and predict how system will perform under heavy loads and to demonstrate and evaluate other non-functional aspect of the system

Capability/Technique Prototypes: Develop, demonstrate, and evaluate a design approach or concept



Forms of Prototypes

Paper Prototypes: Prototype on a paper, an index card , post it notes etc, Storyboards

Screen mockups: Prototypes on wire mesh

Interactive Prototypes: Using high level language using scripting, animation tools

Executable Models: Prototypes which are executable and shows the functionality in working



User Story

What are they and why are they used.

User Story

- A small unit of work
 - Told from a user's perspective
 - Explain desired functionality
 - Crucial part of Agile process
 - Enable conversation and foster collaboration between dev team and customer
-
- ❖ User stories put user at the centre of conversation
 - ❖ User stories provide non - technical context for the development team and their efforts
 - ❖ What they are building and what value the solution will provide



Traditional Requirement vs User Story

Traditional Requirement - written from a System perspective.

- System should be able to do this or A system should allow this

User Story - written from a User perspective

- As a user I want to be able to enter my email so that I can receive notification
- Easy to understand by the business



Questions to ask when writing a User story

Who's your customer

What will satisfy them

What do they **NEED**

User stories help deliver the highest value by focussing on a **SMALL** and **IMMEDIATE** customer need



User Story Format aka Syntax

[WHO] As a <type of user>

It provides perspective

[WHAT] I would like <goal>

Defines the need

[WHY] So that I <reason>

It provides the context



Some User Story Examples

❖ As a Piazza User

I would like the ability to message the instructor directly

So That I can get one on one help for any questions or advice that I need

❖ As a Piazza User

I would like to access the questions other students asked

So that I can self-service and learn beyond the classroom lecture

❖ As a Quercus instructor user

I would like the ability to edit lectures

So that I can keep the course accurate and uptodate



Who Creates the User Story

- ❖ Anybody can write the user story
- ❖ Mostly Product owner or Business Analyst could write the user story

3 C's for Writing User Story

Card



Conversation



Confirmation



- User intent in a sticky note
 - Limits story length and adding details
 - Visualize the whole scope of product/feature
 - A physical overview of value delivery
- Represent that a conversation is due between team. PO, SME
 - Adding details to userstory through backlog refinement, planning etc.
 - Uncover missed scenario and NFR's
- Verify - Acceptance criteria to ensure the user story is implemented correctly.
 - NFR are also covered
 - Includes what needs to be demo-ed



Activity - Write your first User Story

Now that you've learned what user stories are and their format, let's practice writing user stories for booklovers.com. Booklovers is an organization that wants to create a peer-to-peer bookstore. This store will allow for renting, purchasing, and selling of books. Using your imagination and making any necessary assumptions, fill in the blanks for the missing user stories below.

User Story Format

As a user (who)

I want (the why)

So that (some action)

1. REGISTRATION USER STORY

As an unregistered user,

I want to register online

so that I can access the books available to rent, buy and sell



Activity Contd.

1. USER STORY FOR SIGNING IN INTO ONE'S ACCOUNT ON BOOKLOVERS.COM

As a

I want to

so that

2. USER STORY FOR MANAGING ONE'S ACCOUNT ON BOOKLOVERS.COM

As a

I want to

so that



Activity Contd.

3. USER STORY FOR ACCOUNT SETTING ON BOOKLOVERS.COM

As a,

I want to

so that

4. USER STORY FOR ADDING/DELETING ITEMS IN THE CART ON BOOKLOVERS.COM

As a,

I want to

so that

Difference between user stories and requirements



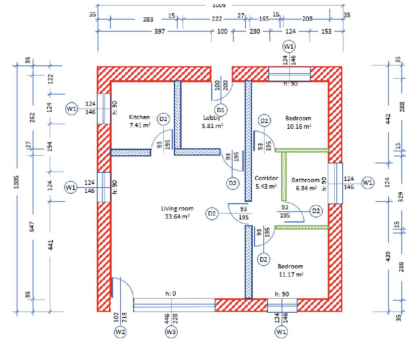
Focus on what user actually need

I should be able to see the profile picture to identify the person



Focus on attributes of the product

User photo to be cropped with 200 x 200 pixel to fit the “profile page”
Focus on what user actually need





Use Case vs User Story

Both are there to describe a goal

User story

- Define User Needs
- Small scope

Use Case

- Define Interaction with System
- Large Scope (Full system)



User Role

- ❖ Group of User - Common needs, interests , expectation , behaviour
- ❖ Brainstorm all the Users

Some of the users that might come to our earlier examples

- **Book Renter**
 - Frequent
 - Occasional
- **Book Buyer**
 - Frequent
 - Occasional
- **Book Seller**
 - Used
 - New



User Personas

Help visualize the kind of person that might be using the product

A short description or biographies of fictitious , hypothetical customer

Should be detailed and include the following

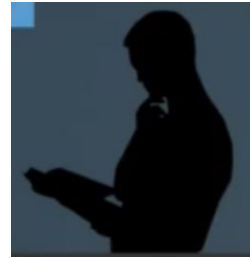
- Name , Photo, Age, Gender
- Personality
- Skills
- Environment (where do they live, what do they usually do etc.)
- What drives or motivates them

Sample Personas



Cathy

- College Graduate
- Lives in an apartment
- Minimalist
- Likes to read



Sam

- School
- Lives with his parents
- Avid reader



Acceptance Criteria

- Set the scope
- Validate the story when it's complete or not
- Condition of satisfaction or user acceptance tests

Given <condition>, **When** < action > , **Then** < result >

Eg. As a User I want to see the delivery status of my message so that I can know that message is received and/or read by my friend.

Given the recipient's mobile is connected to internet , when a message is delivered , Then sender should be able to detect the same using by two tick marks



Acceptance Criteria Contd.

- Acceptance criteria should be automated.
- Acceptance criteria of a technical or enabler user story may NOT be a test case rather artifacts comparing different architecture or mock-up



INVEST

I = Independent

N = Negotiable

V = Valuable

E = Estimable

S = Small

T = Testable



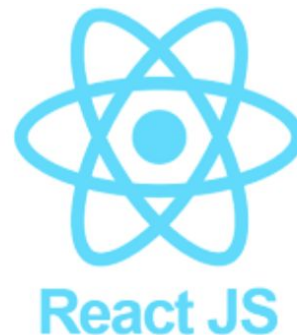
React

a declarative, component-based front-end javascript library developed by Facebook enables developers to build modern, sleek, fast applications with a modular infrastructure

- react is an extremely front-end javascript library/framework!
- With large community support, premade components, libraries, etc

Used widely across industry:

<https://careerkarma.com/blog/companies-that-use-react/>



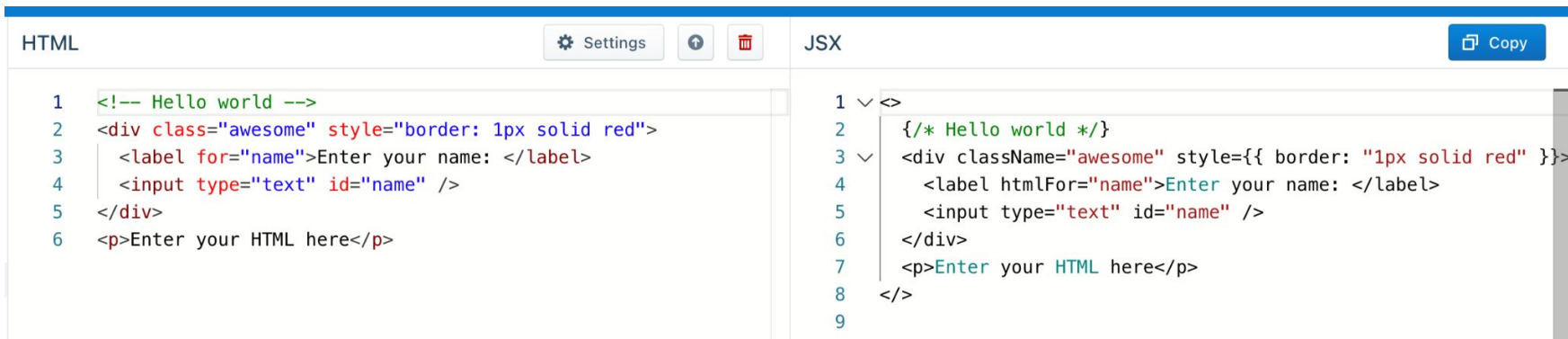
Contd.

React is a JavaScript library for building user interfaces.

- Primarily Javascript and JSX (a 'syntax extension' for Javascript).
- JSX allows you to write HTML code in JavaScript files. It's really simplifies the process of creating interactive web pages.
- Note that you can use plain Javascript (without JSX) to write React code but it's much more difficult/messy
- JSX looks a lot like standard HTML

Let's take a look at an example!





The main benefit of JSX is that it allows you to write an HTML markup inside of JavaScript functions,

<https://transform.tools/html-to-jsx>

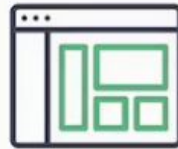
Key Features



JSX



Virtual DOM



Component-based
Architecture

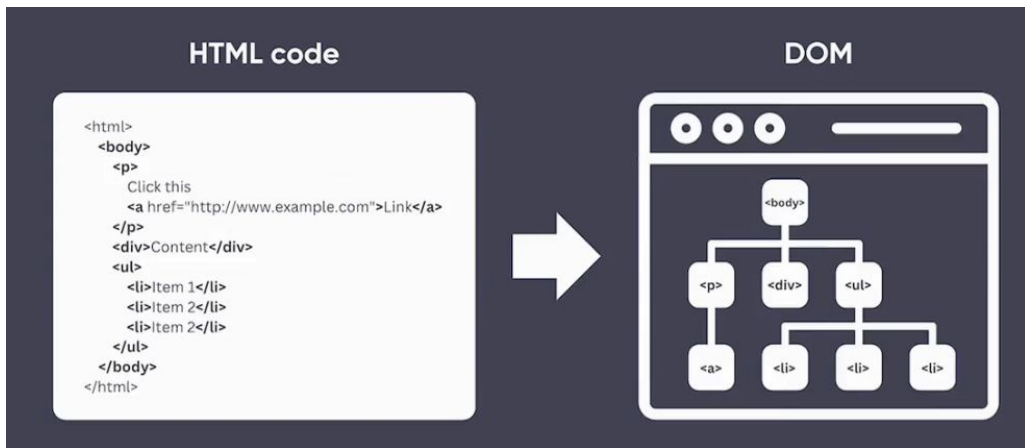


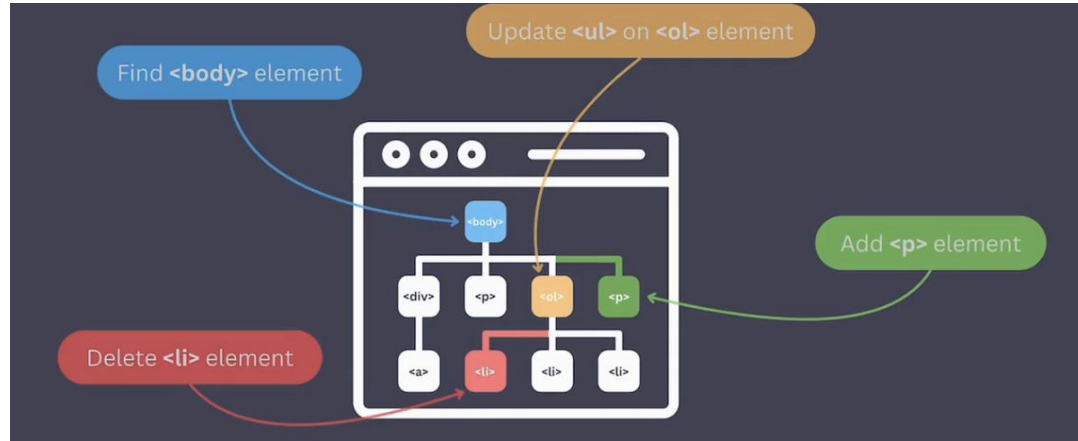
One-Way Data Flow

DOM

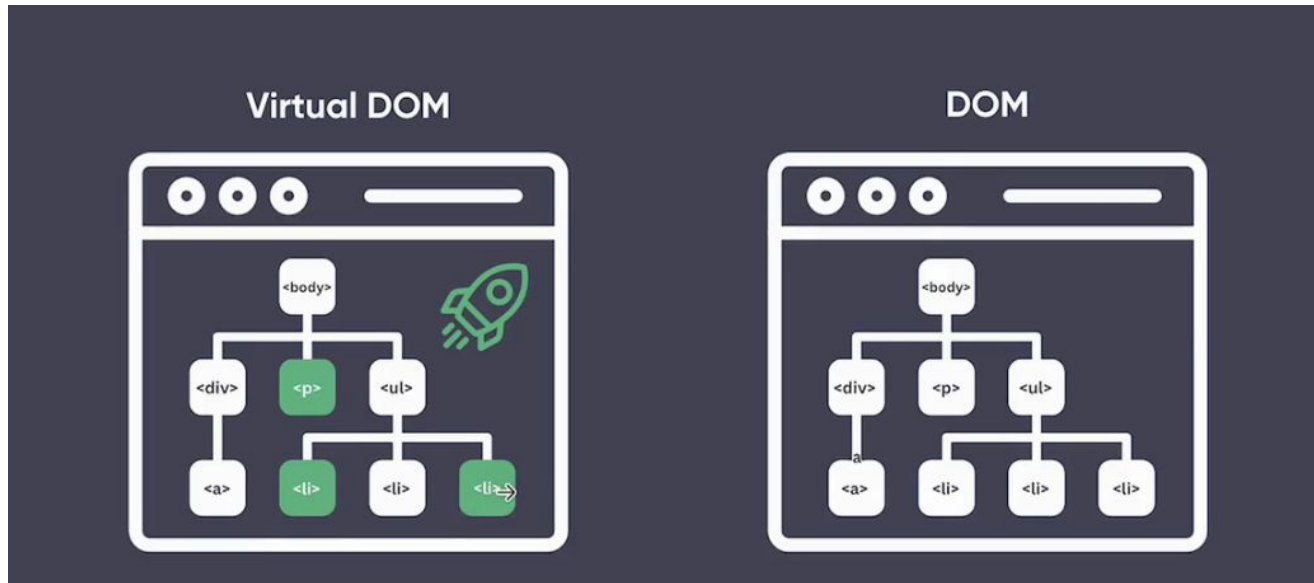
Dom is a document object model. It's needed to organize and access elements on a web page.

It's a JavaScript object that contains all the elements you have in your HTML, like paragraphs, link images, lists, and other elements.





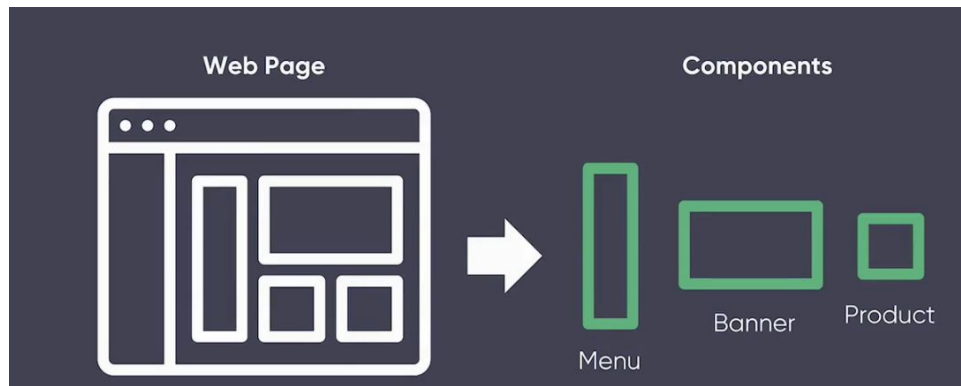
DOM allows use JavaScript to find, add, update and delete elements without reloading the page



All changes in react are first applied to the Virtual DOM. It compares the virtual Dom and real Dom and try to find a difference.

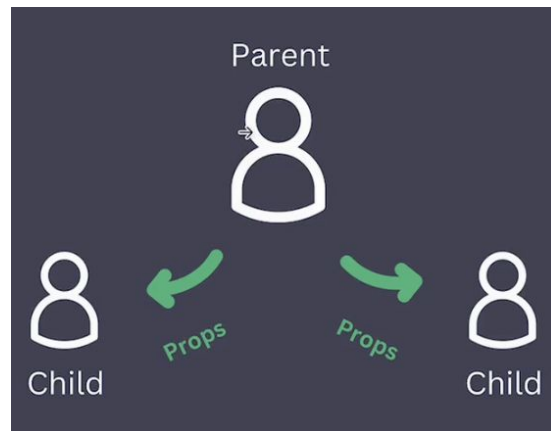
Component Based Architecture

It's a way of building interface, breaking them on a small pieces of code such as button of form or another element on your page.



Each Component can have it's own state and behaviour

One way data flow



Can child components change the data directly?

They must request these changes from the parent through the events, like when the children component



Create React app

Typically you create new react apps using “create react app”, a CLI tool.
Download node.js and install it now if you don't have it.

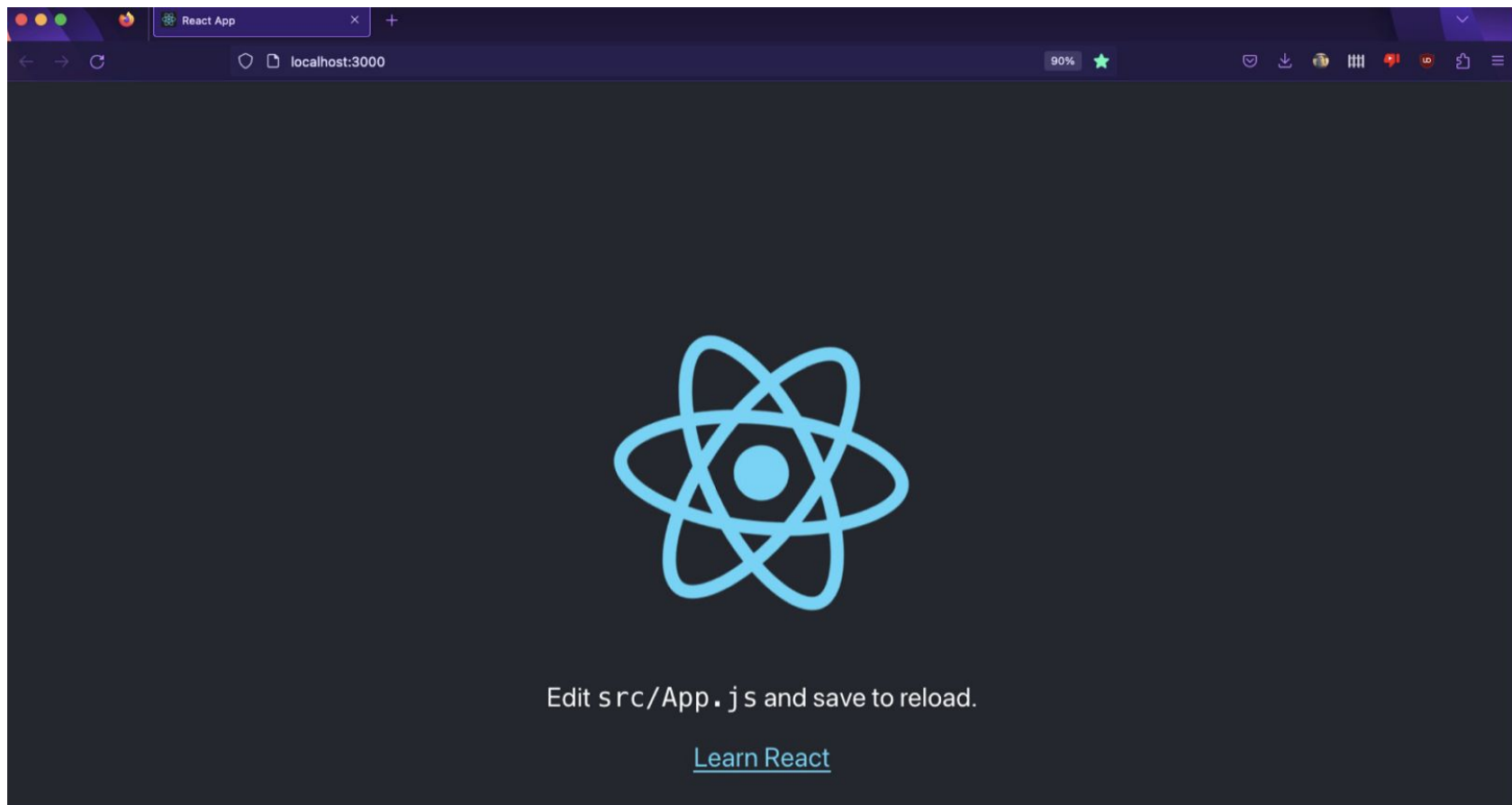
```
$ npx create-react-app your-app-name
```

This step is optional, but this is the command to create a react app. Feel free to run this command - it may take a few minutes.

```
$ cd your-app-name
```

```
$ npm start
```

Let's see what our React app looks like!



Components

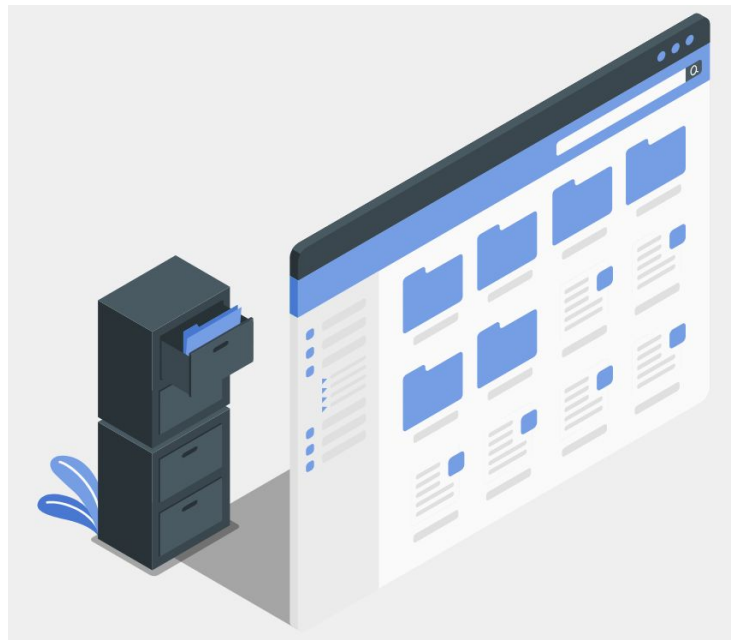
A React **component** is a JavaScript **function** that optionally takes in inputs (props) and **returns ONE JSX element** (this one element can have many children though).

Components are like LEGOs! React pages can be made out of tons of them.

A component is a JavaScript function (or class) that takes in inputs (props/properties) and returns a React element.

They can be purely presentational or stateful (more on that later).

The key to proper component usage is to make them reusable for future use





Javascript Function inside JSX

```
function someFunc(num1, num2) {  
  if (num1 > num2)  
    console.log(num1);  
  
  return num1 + num2;  
}
```

```
const someFunc = (num1, num2) => {  
  if (num1 > num2)  
    console.log(num1);  
  
  return num1 + num2;  
};
```

Think of these as being equivalent function definitions for now. (There are some more [technical differences](#) between these two functions.)

Javascript inside JSX Component

you may have seen us wrap some portions of code in curly braces i.e {...}. Why is that?

here everything outside the '{..}' is JSX, and everything inside is javascript.

if we didn't have the curly branches there, our javascript code would be interpreted as a string and NOT code (ie "messages.reduce((prev, curr) => prev.concat(curr), "")")

```
<h1>
  {
    |   messages.reduce((prev, curr) => prev.concat(curr), "")
    |
  }
</h1>
```

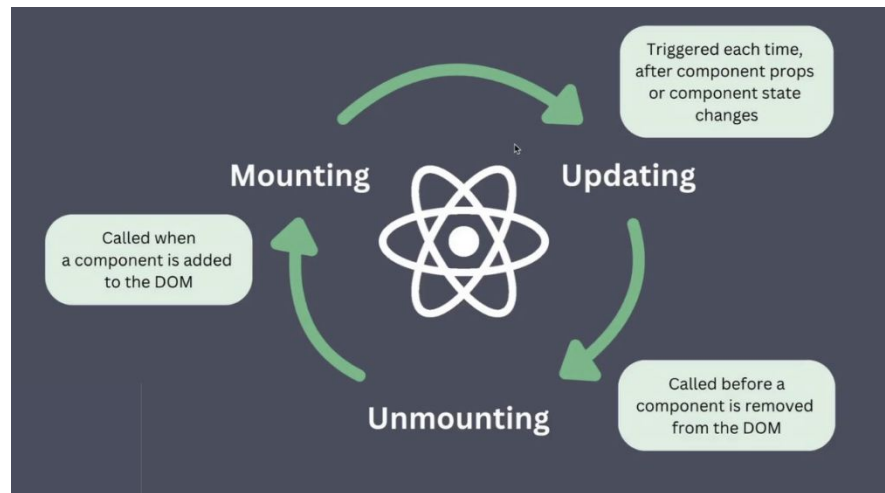


Passing Props to Component

- React components use props to communicate with each other, and every parent component can share some data or some information with the child component by giving them props.
- Props are very similar to HTML attributes, but they allow you to pass any JavaScript value like text, numbers, objects, arrays, or even functions.
- Array component are function , and it means you can also pass react component as a prop.

Component State & Lifecycle

- Component state is a special JavaScript object that can store data inside of component.
- Also, component state can be changed during the lifecycle of the component, and also one component is unmounted, the state is destroyed





Hooks

Introducing Hooks!

React Hooks are ways to “hook” into the component lifecycle and call specific methods for any state!

These methods become super useful when the component needs to fetch data on load (hint hint) or perform actions on the page

Some examples of hooks include:

- `useState`
- `useEffect`
- `useRef`
- `useCallback`

useState

this react hook allows us to persist data across re-renders (and forces a re-render when our state changes) in that state!

In the example function you can create a count state for the component using useState as shown.

You can use count to access count within the component.

```
function Example() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>You clicked {count} times</p>  
      <button onClick={() => setCount(count + 1)}>  
        Click me  
      </button>  
    </div>  
  )  
}
```




useRef

useRef is a React Hook that lets you reference a value that's not needed for rendering.

useRef(initialValue)

```
import { useRef } from 'react';

function MyComponent() {
  const intervalRef = useRef(0);
  const inputRef = useRef(null);
  // ...
```

```
1  import { useRef, useState } from 'react';
2
3  interface DemoProps {}
4
5  export default function Demo({}: DemoProps) {
6    const [count, setCount] = useState(0);
7    const countRef = useRef(0);
8
9    const handleIncrement = () => {
10      setCount(count + 1);
11      countRef.current++;
12
13      console.log('State:', count);
14      console.log('Ref:', countRef.current);
15    };
16
17    return (
18      <div className='tutorial'>
19        Count: {count}
20        <button onClick={handleIncrement}>Increment</button>
21      </div>
22    );
23  }
24
```



```
1  import { useRef, useState } from 'react';
2
3  interface DemoProps {}
4
5  export default function Demo({}: DemoProps) {
6    const [count, setCount] = useState(0);
7    const countRef = useRef(0);
8
9    const handleIncrement = () => {
10      // setCount(count + 1);
11      countRef.current++;
12
13      // console.log('State:', count);
14      console.log('Ref:', countRef.current);
15    };
16
17    return (
18      <div className='tutorial'>
19        Count: {countRef.current}
20        <button onClick={handleIncrement}>Increment</button>
21      </div>
22    );
23  }
24
```



Prompting an AI for React Code

Be Specific about React Patterns

- Instead of “make button” ,say “create functional React component with useState that toggles between ‘Start’ and ‘Stop’ text when clicked”.

Specify the React version and features you want

- Mention using “React 18 features” or “React 19 features” matching the project you are working in.

Include your existing component structure and provide guidance

- Share your current Counter Component code so that AI can built upon it rather than rewriting everything

Ask for complete, working components

- Request a “complete functional component that I can copy and paste” rather than just code snippets that need integration.

Specify state Management approach

- Be clear whether you want local useState, useReducer, or other state solution for more complex features.



Thank You